

Tag 6

Listen-Wiederholung & Entscheidungen treffen

Python-Grundlagen-Schulung

Was du heute lernst

1. Kurze Auffrischung: Was konnten Listen nochmal?
2. Wie trifft mein Programm **Entscheidungen**? (`if` / `elif` / `else`)
3. Wie **vergleiche** ich Werte miteinander?
4. Wie **kombiniere** ich mehrere Bedingungen? (`and` / `or` / `not`)

Teil 1

Kurze Listen-Auffrischung

Was du schon über Listen weißt

Eine Liste ist wie eine **Kiste mit mehreren Fächern**, in der du mehrere Werte hintereinander speicherst.

```
offene_ports = [80, 443, 22]
```

Du kannst Werte **hinzufügen**, **ändern**, **entfernen** und **sortieren**.

Ein Beispiel zum Aufwärmen

```
offene_ports = [80, 443, 22]
offene_ports.append(8080)
print(f"Überwachte Ports: {len(offene_ports)}")
```

`.append(...)` fügt einen Wert **hinten** an

`len(...)` verrät dir, **wie viele** Werte in der Liste sind

Heute im Übungsblock

6 Aufgaben zum Warmwerden, unter anderem:

- Werte hinzufügen und ersetzen
- Minimum und Maximum finden
- Einen Wert entfernen und speichern
- Sortieren
- Prüfen: Ist ein Wert in der Liste enthalten?

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Teil 2

Entscheidungen treffen

Bisher: Dein Code lief stur ab

Zeile 1 → Zeile 2 → Zeile 3 → ...

Immer in der gleichen Reihenfolge,
egal was passiert.

Das nennt man **sequenziellen Ablauf**.

Ab heute: Dein Code kann reagieren!

"Ist der Speicher voll? → Sende eine Warnung."

"Ist er es nicht? → Mache normal weiter."

Genau das ermöglichen **Bedingungen**.

Bild: Die Weggabelung

Ein Wanderer kommt an eine Weggabelung.

**"Ist der linke Weg frei? Wenn ja, gehe links.
Sonst gehe rechts."**

Eine `if`-Bedingung ist genau so eine Weggabelung
in deinem Code.

Warum ist das so wichtig?

Ohne Bedingungen ist ein Programm nur eine
Liste stur abzuarbeitender Befehle.

Mit Bedingungen wird es zu einem
System, das auf seine Umgebung **reagiert.**

Ein einfaches Beispiel

```
speicher = 8 # GB freier Speicher  
  
if speicher < 10:  
    print("Achtung: Wenig Speicherplatz!")  
else:  
    print("Speicherplatz ist ausreichend.")
```

speicher ist 8 → 8 < 10 ist **wahr**

→ die erste Zeile wird ausgeführt

Die einzelnen Bausteine

if → "wenn" – leitet die Bedingung ein

Bedingung (z. B. `speicher < 10`) → ergibt `True` oder `False`

: → Doppelpunkt, **immer** nötig nach `if / else`

Einrückung → zeigt, was zum `if`-Block gehört

else → "andernfalls" – optional, fängt den Rest ab

Ganz wichtig: die Einrückung

Anders als in vielen anderen Programmiersprachen
benutzt Python **keine geschweiften Klammern** `{}` .

Stattdessen zeigt die **Einrückung selbst**,
was zusammengehört.

Alle Zeilen im selben Block müssen
gleich weit eingerückt sein!

Ein wichtiger Merksatz

**Python prüft die Bedingung
genau EINMAL,
an genau dieser Stelle im Code.**

Vergleichsoperatoren

Das Vokabular der Bedingungen

Die sechs Vergleichsoperatoren

Zeichen	Bedeutung
<code>==</code>	ist gleich
<code>!=</code>	ist ungleich
<code><</code>	kleiner als
<code>></code>	größer als
<code><=</code>	kleiner oder gleich
<code>>=</code>	größer oder gleich

Die wichtigste Stolperfalle

`=` → Zuweisung

"Speichere den Wert in der Variable"

`==` → Vergleich

"Sind diese beiden Werte gleich?"

Ein einzelnes statt doppeltes `=` in einer Bedingung
ist einer der häufigsten Anfängerfehler!



Teil 3

Mehrere Bedingungen kombinieren

Wenn eine Bedingung nicht reicht

Ein Server ist vielleicht nicht schon dann kritisch,
wenn **nur** die CPU hoch ist.

Vielleicht erst, wenn **gleichzeitig**
auch der Arbeitsspeicher knapp wird.

Dafür brauchen wir **logische Operatoren**.

and: beide müssen zutreffen



Bild: Eine Tür mit **zwei Schlössern**.

Beide müssen **gleichzeitig** aufgeschlossen sein,
damit die Tür aufgeht.

```
if cpu_auslastung > 80 and ram_auslastung > 80:  
    print("Kritisch: Gesamter Server ist überlastet!")
```

or: einer reicht schon

  Bild: Eine Tür mit zwei Klingelknöpfen.

Es reicht, wenn **irgendjemand** klingelt,
damit die Glocke ertönt.

```
if cpu_auslastung > 80 or ram_auslastung > 80:  
    print("Warnung: Eine Komponente ist stark ausgelastet.")
```

not: das Gegenteil

`not` dreht einen Wahrheitswert einfach um.

Aus `True` wird `False` .

Aus `False` wird `True` .

```
if not gesperrt:  
    print("Das System ist nicht gesperrt.")
```

Die Merktabelle

A	B	A and B	A or B
wahr	wahr	wahr	wahr
wahr	falsch	falsch	wahr
falsch	wahr	falsch	wahr
falsch	falsch	falsch	falsch

in: Ist etwas in einer Liste enthalten?

```
vip_kunden = ["Müller", "Schmidt", "Fischer"]  
kunde = "Schmidt"  
  
if kunde in vip_kunden:  
    print("Priorisierter VIP-Support aktivieren.")  
else:  
    print("Standard-Support-Warteschlange.")
```

`in` prüft: **Kommt dieser Wert in der Liste vor?**

Viel übersichtlicher als

```
kunde == "Müller" or kunde == "Schmidt" or ...
```

Bonus

Mehr als zwei Fälle: elif

Wenn zwei Fälle nicht reichen

`elif` heißt kurz für "**else if**" – "sonst wenn".

Damit prüfst du **mehrere** Fälle nacheinander.

Python nimmt den **ersten** Fall, der zutrifft –
alle weiteren werden übersprungen.

Beispiel: Temperaturalarm

```
temp = float(input("Temperatur (°C): "))

if temp > 25:
    print("Kritisch: Zu heiß!")
elif temp < 18:
    print("Warnung: Zu kalt!")
else:
    print("Temperatur optimal.")
```

Python prüft **von oben nach unten**:
erst `> 25` , dann `< 18` , sonst `else` .

Jetzt du: der große Übungsblock

Was dich erwartet

Kategorie A – Einfache Bedingungen

Festplatten-Wächter, Passwort-Prüfer, Port-Checker

Kategorie B – mehrere Fälle mit elif

Temperaturalarm, Ping-Bewertung, Ticket-Priorität

Kategorie C – Kombinierte Logik

IP-Firewall, Login mit Wartungsmodus, Multi-Faktor-Zugang

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Ein Beispiel zum Warmwerden

```
korrektes_passwort = "admin123"  
eingabe = input("Passwort eingeben: ")  
  
if eingabe == korrektes_passwort:  
    print("Zugriff gewährt")  
else:  
    print("Zugriff verweigert")
```

Ein Vergleich mit `==` , verpackt in ein `if / else` –
genau das Grundmuster von heute.

Zum Schluss

Typischer Anfängerfehler

Uneinheitliche Einrückung

```
if speicher < 10:  
    print("Wenig Speicher!")  
    print("Bitte prüfen.")    # ✗ falsch eingerückt
```

✗ IndentationError

Alle Zeilen im selben Block müssen
exakt gleich weit eingerückt sein –
nicht mal 4, mal 6 Leerzeichen mischen!

Was du heute gelernt hast

- ✓ `if` / `elif` / `else` lassen dein Programm **entscheiden**
- ✓ Vergleichsoperatoren wie `==` , `<` , `>` liefern `True` oder `False`
- ✓ `and` , `or` , `not` verknüpfen mehrere Bedingungen
- ✓ `in` prüft, ob ein Wert in einer Liste enthalten ist

Morgen: Fehler finden & die `while`-Schleife 🎉